Differentiating Human Users from Malicious Simulated Interaction Using Keystroke and Mouse Dynamics

Uail Zhukenov Karen Nurlybekov

April 15, 2026

Abstract

This research investigates whether behavioral patterns derived from keystroke and mouse dynamics can reliably distinguish genuine human interaction from malicious simulated behavior. By extracting behavioral features from publicly available datasets and evaluating a range of machine learning models, the study assesses the discriminative power of movement variability, speed statistics, and typing rhythm characteristics. The research further explores multi-modal fusion, combining keystroke and mouse channels to raise the cost of attack for potential adversaries. Results are analyzed across both in-distribution and out-of-distribution settings, and a partial attack analysis is conducted to evaluate system robustness under realistic threat scenarios. Additionally, the computational efficiency is assessed to determine their suitability for real-time deployment in high-traffic production environments.

1 Introduction

With the rapid advancement of technology, distinguishing between genuine human users and malicious automated behavior—such as bots, scripted interactions, or replay-based attacks—has become increasingly important. Traditional security mechanisms, including passwords and CAPTCHAs, are often intrusive, inconvenient for users, or vulnerable to circumvention by increasingly sophisticated automation. As a result, there is growing interest in passive and continuous security mechanisms that can differentiate humans from automated agents without disrupting the user experience. This quantitative research investigates whether behavioral patterns derived from keystroke and mouse dynamics can reliably distinguish genuine human interaction from malicious simulated behavior.

Recent studies suggest that behavioral biometrics, particularly mouse dynamics, are effective for detecting automated interactions. Acien et al. proposed BeCAPTCHA-Mouse, a framework that leverages neuromotor-inspired mouse features and both real and synthetically generated trajectories to distinguish human users from bots, achieving high detection accuracy even against highly realistic automated behavior [1]. These results indicate that subtle characteristics of human motor control are difficult for automated systems to replicate, making mouse-based behavioral analysis a promising alternative to traditional CAPTCHA mechanisms. However, this task is complicated by the emergence of session-replay bots, which attempt to evade detection by replaying previously recorded human interaction data. Sadeghpour and Vlajic demonstrate that while human interactions naturally exhibit micro-variations over time, replayed sessions often lack the dynamic variability inherent in live behavior [3]. Their findings highlight the importance of modeling temporal and behavioral variability when trying to distinguish genuine human interaction from malicious simulated activity.

In parallel, extensive research has explored keystroke and mouse dynamics for user authentication, often employing machine learning models such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks, with reported accuracies reaching up to 99% [2]. Although these studies demonstrate the power of behavioral biometrics, they primarily focus on distinguishing between individual users rather than differentiating human behavior from automated or simulated interactions. Consequently, the problem of detecting non-human behavior, particularly replay-based or algorithmically generated interactions, remains underexplored. In contrast, this research shifts the focus toward identifying fundamental behavioral characteristics that distinguish live human input from malicious simulated behavior by analyzing both keystroke and mouse dynamics. By examining the variability and temporal structure of human interaction patterns, this work aims to contribute

toward more robust authentication mechanisms.

Thus, this study aims to determine whether machine learning models can identify patterns that are inherently difficult for automated systems to replicate, thereby contributing to the development of non-intrusive and continuous security mechanisms for detecting malicious automated activity. To achieve this goal, the research will pursue the following objectives:

1. Pre-process and structure the collected data, including cleaning, organizing sessions, and labeling interactions as human or simulated or malicious-like.
2. Extract meaningful behavioral features from keystroke and mouse dynamics, such as timing intervals, movement trajectories, and interaction variability.
3. Train and evaluate machine learning models capable of distinguishing between genuine human interaction and simulated behavior using the extracted behavioral features.
4. Analyze the performance of the model and the importance of features to identify which behavioral characteristics most strongly differentiate human input from malicious automated activity.

2 Literature Review

One relevant study in behavioral bot detection examines how modern machine learning techniques can successfully bypass many CAPTCHA systems. For example, the study shows that even Google reCAPTCHA v3 can be defeated by generating realistic mouse trajectories using reinforcement learning methods. Because of this vulnerability, the authors emphasize the need for more reliable approaches to detect bots and protect users from malicious automated software. The proposed method uses a passive bot detection approach based on mouse dynamics, which analyzes natural patterns in human mouse movements that result from neuromotor behavior. To support this approach, the study generates realistic synthetic mouse trajectories using heuristic functions and data-driven techniques such as Generative Adversarial Networks (GANs). These synthetic trajectories are then used to train machine learning models that can distinguish between human and automated behavior. The study also introduces a dataset containing thousands of mouse movement trajectories collected from multiple users performing controlled interaction tasks. The dataset includes both human and synthetic mouse activity, making it useful for training and evaluating bot detection systems. Although this dataset would be highly valuable for the purposes of our research because it provides detailed labeled trajectory data, access to the full dataset is limited; therefore, this study relies on an alternative publicly available dataset [1].

Another study explores the problem of session-replay bots, a newer type of automated attack that is difficult to detect because it replays previously recorded user interactions [3]. These attacks make it challenging to determine whether two very similar sessions were created by the same human user or simply reproduced by a bot. To support research in this area, the study introduced the ReMouse dataset, which was collected in a controlled environment. In this dataset, 100 participants from different geographic locations completed several rounds of a simple online game called "Catch Me If You Can!" through an interactive web platform. Each participant performed multiple sessions that included predefined actions such as left-click, right-click, and drag-and-drop operations.

The study also reviews several existing mouse dynamics datasets, including Balabit, Bogazici, Attentive Cursor, SapiMouse, Chao Shen, and DFL, and compares them in a table format [3]. It highlights that many of these datasets do not include repeated user sessions, which limits their usefulness for studying session-replay attacks. To analyze the ReMouse dataset, the researchers applied both statistical methods and machine learning approaches, including deep convolutional neural networks and unsupervised feature learning. The results show that even when the same user repeats a session, small variations in mouse behavior still occur. At the same time, sessions produced by different users remain sufficiently distinct. This suggests that session-replay bot attacks can still be detected by analyzing behavioral mouse patterns. Overall, the dataset provides repeated human sessions, a diverse group of participants, and detailed mouse interaction features, making it valuable for research in behavioral biometrics and advanced bot detection [3]. However, although this dataset would be highly useful for the purposes of this research, it could not be obtained because access requires additional steps such as approvals or financial costs.

Another work on behavioral biometrics demonstrates the advantages of using a multi-modal approach that combines multiple types of user interaction data instead of relying on a single behavioral signal [2]. Rather than analyzing only keystroke dynamics or only mouse movements, the approach integrates both sources of information. This allows the system to capture a more complete representation

of how a user interacts with a computer. The study shows that combining different behavioral signals through temporal alignment helps preserve the order and context of user actions. Since user interactions occur as sequences over time, maintaining this structure enables machine learning models to better understand behavioral patterns. It is also demonstrated that when these behavioral signals are integrated, deep learning models such as multilayer perceptrons (MLPs), convolutional neural networks (CNNs), and long short-term memory networks (LSTMs) can learn more complex patterns in user behavior. As a result, these models are more capable of identifying unusual or suspicious activity that might not be detected when keystrokes or mouse movements are analyzed separately. Although the dataset used in that study is not used in our research, the methodology provides a useful conceptual framework for developing multi-modal behavioral biometric systems [2]. By emphasizing the relationship between different types of user interactions over time, the work supports the idea that combining multiple behavioral signals can improve the reliability and accuracy of behavioral biometric classification.

An additional study on keystroke-dynamics authentication examines how secure such systems are when facing automated forgery attacks [4]. The research introduces a remote authentication framework called TUBA, which collects timing information from a user's typing and applies a Support Vector Machine (SVM) model to classify user behavior. Several experimental scenarios were evaluated in the study [4]. One scenario compared typing patterns between different human users (human vs. human), while another focused on detecting automated attacks where bots attempt to imitate human typing behavior (human vs. bot). To simulate these attacks, two bot models called GaussianBot and NoiseBot were developed. These bots generate typing patterns that are statistically similar to real human typing data. The results demonstrate that keystroke dynamics can effectively distinguish legitimate users from both other individuals and automated forgeries while maintaining low false positive rates. For the purposes of our research, this work is particularly useful because it clearly explains the feature extraction and classification techniques used in keystroke-dynamics systems. Reviewing this study helped provide a better understanding of how typing behavior can be modeled and analyzed for authentication and bot detection.

A publicly available dataset used in this research provides an important benchmark for studying keystroke-based behavioral biometrics [5]. The dataset contains human-written free-text typing samples together with synthetic versions of the same keystroke sequences generated using different forgery techniques and varying levels of attacker knowledge. The human typing samples were collected from three previously published datasets, while the synthetic samples were created to imitate legitimate users' typing behavior under different attack scenarios. The dataset records detailed timing features such as hold time (the duration a key is pressed) and flight time (the time interval between keystrokes), which allows researchers to analyze typing rhythm patterns. Because the dataset includes both genuine and forged samples that share identical keystroke sequences, it is particularly useful for evaluating machine learning models that aim to distinguish real human typing from artificially generated behavior. For this reason, it was selected as the primary source of keystroke dynamics data for this research [5].

This dataset was developed in connection with research that investigates liveness detection in keystroke dynamics, which focuses on identifying synthetic typing forgeries [6]. The study argues that traditional keystroke authentication systems are often evaluated using overly simple attack models, such as zero-effort impostors, which do not accurately represent real-world threats. To address this limitation, the research introduces several spoofing techniques based on statistical modeling of keystroke timing patterns and evaluates a liveness detection approach designed to distinguish legitimate typing behavior from synthetic samples. The dataset used in this study was specifically created to support this research by providing both human and synthesized typing data generated under different attacker knowledge conditions. As a result, it offers a realistic environment for analyzing spoofing attacks and testing defenses in behavioral biometric systems, making it highly relevant to the objectives of this work [6].

A study on web bot detection proposes a framework that combines web log analysis with mouse behavioral biometrics to identify automated visitors on websites [7]. The approach is based on the idea that advanced bots can mimic many browser behaviors and even appear similar to real users, making them difficult to detect using traditional methods. To address this problem, the study introduces a detection framework consisting of two complementary modules: one that analyzes web access logs (such as page requests, browsing patterns, and HTTP errors) and another that analyzes mouse movement behavior. Each module produces a classification score indicating whether a visitor is likely to be a human or a bot, and these scores are combined to improve the overall detection accuracy. This multi-source approach allows the system to capture both the temporal characteris-

tics of browsing behavior and the spatial characteristics of mouse movements, leading to more robust bot detection [7].

To support research in this area, a dataset containing mouse dynamics and web interaction data was created and later made publicly available [8]. The dataset includes recorded mouse movements and interaction logs from website sessions, providing detailed behavioral data that can be used to study differences between human users and automated bots. Because it contains realistic user interaction data along with behavioral features extracted from web activity, the dataset provides a valuable resource for evaluating machine learning models for bot detection. Due to its close connection with the proposed detection framework and its detailed behavioral interaction records, this dataset is highly relevant for our research on bot detection, and it can serve as a useful benchmark for experiments involving mouse dynamics and user interaction analysis [7, 8].

3 Dataset

3.1 Mouse Dynamics Dataset (M4D)

For the mouse dynamics component, this study uses the M4D (Mouse for Detection) dataset, a publicly available dataset designed specifically for mouse-based bot detection research [8]. The dataset consists of 150 mouse trajectory sessions, evenly distributed across three classes: human users, moderate bots, and advanced bots. Each session contains a sequence of cursor coordinates.

Human-generated sessions typically include approximately 10,000 mouse events and exhibit curved and irregular trajectories, reflecting natural neuromotor behavior. In contrast, bot-generated sessions contain between 3,000 and 7,000 events and tend to produce straighter, more uniform movement patterns. These differences in trajectory shape and event distribution provide a basis for distinguishing between human and automated behavior.

However, an important limitation of this dataset must be acknowledged. Bot trajectories are generated using fixed pixel step sizes (either 1 pixel or 8 pixels per event), resulting in almost zero variation in movement speed. Consequently, features such as the standard deviation of speed become highly discriminative. A model can therefore achieve near-perfect classification performance by learning a simple rule (e.g., near-zero speed variation indicates a bot), rather than capturing more complex behavioral differences. As a result, performance metrics obtained from this dataset may be artificially inflated and should be interpreted with caution.

The key characteristics of the Mouse Dynamics Dataset, including the specific step-sizes for bot variants and the training/testing split, are presented in Table 1.

Table 1: M4D Composition and Experimental Setup

Property	**Value**
Total sessions	150
Human sessions	50
Advanced bot sessions	50 (8 px steps)
Moderate bot sessions	50 (1 px steps)
Data format	(x, y, timestamp)
Train / test split	80% / 20%
Test set size	$\sim$30 sessions

3.2 Keystroke Dynamics: Use of PROSODY, LSIA, and KM

For the keystroke dynamics component, this study uses the dataset introduced by González and Calot [5]. This dataset combines keystroke recordings from three independent sources: LSIA, KM (Killourhy–Maxion), and PROSODY. These datasets differ in language, task type, and collection conditions, providing a diverse representation of typing behavior. Each human typing sample is paired with multiple synthetic versions generated using different forgery techniques, including Average, Gaussian, Histogram, Non-Stationary Histogram, and Uniform methods. Additionally, six custom bot models of varying complexity were generated for this study, resulting in a large dataset containing around 470,400 synthetic samples and around 18,816 human samples. Each record includes three features: virtual key code (VK), hold time (HT), and flight time (FT).

To ensure robust model generalization, data partitioning was conducted at the participant level rather than the session level. This hierarchical split is essential to prevent data leakage; if sessions from the same individual were present in both training and testing sets, the classifier might achieve high accuracy by simply learning typing signatures of specific users rather than identifying behavioral differences between humans and bots. Hence, the dataset, comprising 488 participants from the PROSODY sub-dataset (GAY, GUN, and REVIEW), was divided into four disjoint subsets (Table 2).

The decision to isolate bot templates from the training pool is a critical design choice. If the synthetic agents were calibrated using human profiles already known to the model, the classifier could theoretically succeed by identifying which specific individual was being imitated. By utilizing a separate pool of participants for bot generation, we

force the model to rely on automation-specific behavioral markers rather than familiarity with individual user profiles.

Similarly, the Calibration set (15 percent) was held out to ensure that the mapping of raw classifier outputs probabilities remains unbiased. Raw scores (e.g., a value of 0.8) often lack probabilistic meaning; this step ensures the model is well-calibrated on unseen data.

To assess system's portability, an additional 156 participants from the LSIA (n=136) and KM (n=20) were reserved for out-of-distribution testing. These datasets involve different languages, task contexts, and text formats. This OOD evaluation serves as a stress test for the model's ability to generalize beyond the specific lexical and environmental constraints of the training data.

4 Exploratory Data Analysis

4.1 Trajectory and Timing Analysis

Before training the models, exploratory data analysis (EDA) was conducted to better understand the differences between human and bot behavior using trajectory and timing analysis. The analysis revealed clear visual and statistical differences between classes. Human trajectories tend to be curved and irregular, while bot trajectories are more linear and consistent, which can be clearly seen in Figure 1 .

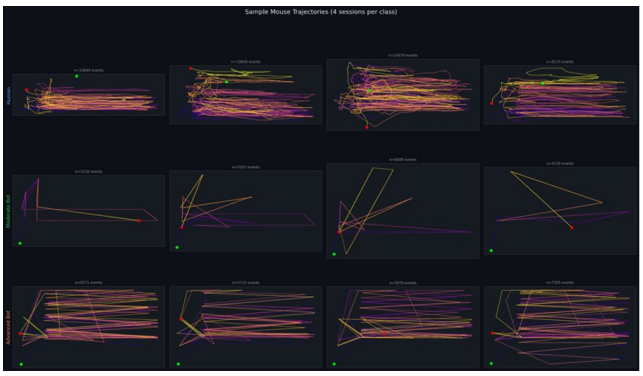

Figure 1: Mouse trajectory examples for human (left), moderate bot (centre), and advanced bot (right). Human paths are curved and irregular. Bot paths are straight or mechanically regular.

Timing analysis showed that human interactions exhibit natural variability in event intervals, whereas bots produce either highly consistent or artificially irregular timing patterns. Additionally, another crucial finding is that the coefficient of variation (CV) of step distance is close to zero for bot-generated data but significantly higher for human data. This feature alone provides a strong signal for classification and largely explains the high performance observed in the models. Refer to Figures 2 and 3 for visualization.

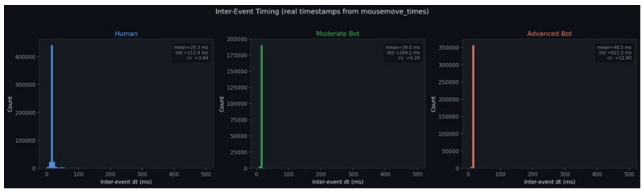

Figure 2: Inter-event timing distributions for human and bot interactions. Advanced bots exhibit frequent, irregular pauses; moderate bots show high consistency; and human users demonstrate natural rhythmic variation.

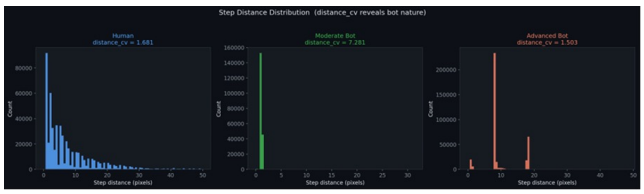

Figure 3: Distribution of cursor step distances. Automated sessions show narrow spikes at fixed pixel intervals, whereas human sessions exhibit a broad spectrum of movement distances.

4.2 Typing Rhythm Variations: Fixed-Text vs. Free-Text

The analysis of keystroke datasets highlighted important differences between sub-datasets. In particular, KM (fixed-text password typing) exhibits different timing distributions compared to PROSODY (free-text typing). This is expected, as typing a memorized password involves different cognitive and motor processes than composing text. These differences are important when evaluating model generalization. Additionally, bot-generated keystroke data tends to show either overly consistent timing (in simpler bots) or statistically similar averages without natural variability (in more advanced bots). The most sophisticated bot models (e.g., KDEBot and HMMBot) are the most difficult to distinguish from human users, as they better approximate real typing distributions. A comparative overview of keystroke data is provided in Figure 4, which illustrates session counts and timing distributions across various subsets, highlighting the unique profiles of fixed-text versus free-text conditions.

Table 2: PROSODY Data Distribution and Subset Functions

Subset	Participants (n)	Percentage	Primary Function
Training	170	35%	Model training (Human vs. Synthetic)
Bot Templates	122	25%	Source material for generating bot behavior
Calibration	73	15%	Probability calibration and scaling
In-Distribution Test	123	25%	Evaluation on known language and text domains

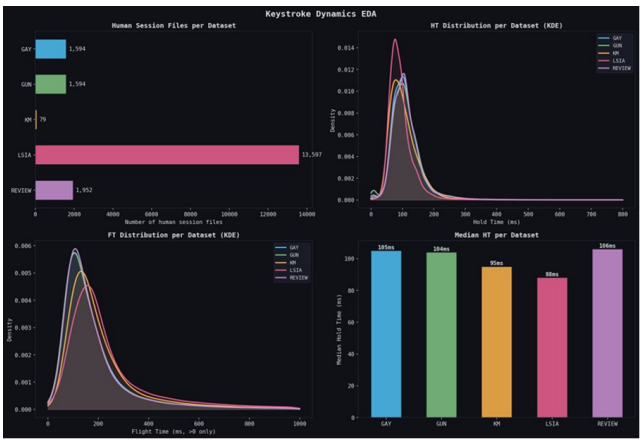

Figure 4: Top-left: session counts across subsets. Top-right: Hold Time (HT) KDE plots. Bottom-left: Flight Time (FT) distributions. Bottom-right: Median HT comparison. Note the distinct distributional shifts in the KM (fixed-text) condition relative to free-text PROSODY samples.

5 Methodology

5.1 Feature Engineering

5.1.1 Mouse Movement Features

To move beyond simple detection patterns such as identifying straight mouse trajectories, the feature engineering process focuses on input-agnostic behavioral characteristics that describe how a user interacts with the system rather than what specific task is being performed.

Total of 26 features were initially extracted from each trajectory. These features capture multiple aspects of movement, including speed, acceleration, smoothness, spatial extent, and timing behavior. Together, they provide a comprehensive representation of cursor movement patterns. To reduce redundancy, two feature selection techniques were applied. First, highly correlated features were removed through de-correlation analysis. Second, Recursive Feature Elimination with Cross-Validation (RFECV) was used to identify the smallest subset of features that maintains high performance. RFECV iteratively removes less important features while evaluating model performance using 5-fold cross-validation. The results showed that only a small number of features were needed to achieve near-perfect performance. RFECV selected features such as standard deviation of speed (std speed), standard deviation of acceleration (std acceleration) and horizontal range (x range) were ultimately chosen, as they provided sufficient power. Refer to Figure 5 for comparison.

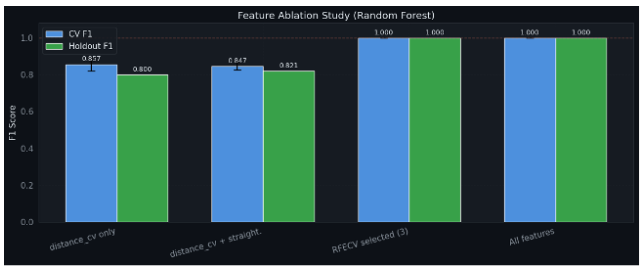

Figure 5: Evaluation of model performance across different feature subsets. The results indicate that a reduced set of three selected features achieves performance parity with the full 26-feature set, suggesting high feature redundancy in the mouse dynamics dataset.

5.1.2 Keystroke Burst-Pause Features

Human typing naturally alternates between bursts of rapid keystrokes and pauses associated with cognitive processes such as thinking or revising. In contrast, bots typically lack this structured rhythm. A pause was defined as any flight time (FT) greater than or equal to 300 milliseconds, corresponding to approximately the 87th percentile of the dataset. Based on this definition, seven features were extracted to capture burst and pause characteristics, including burst frequency, burst length statistics, and pause duration metrics. These features are presented in Table 3.

Correlation analysis showed that while some features are moderately related, none are strongly redundant, indicating that each contributes meaningful information to the model. Refer to 6.

Table 3: Keystroke Dynamics Features

Feature	What It Measures
burst_count	Number of bursts in the session—how many times typing restarts after a pause.
burst_mean_len	Average number of keystrokes per burst.
burst_std_len	Variation in burst lengths—humans typically exhibit irregular burst sizes.
burst_max_len	Length of the longest recorded burst in the session.
pause_rate	Fraction of inter-key gaps categorized as pauses ($\geq$ 300 ms).
pause_mean_dur	Average duration of all detected pauses in milliseconds.
pause_std_dur	Variation in pause durations—humans tend to pause for varying lengths of time.

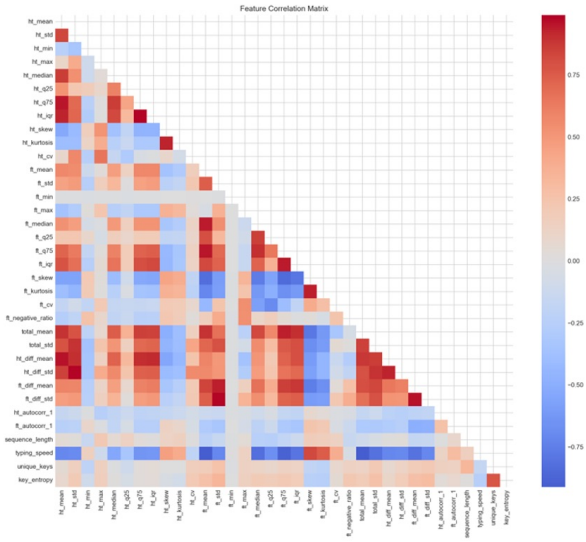

Figure 6: Correlation matrix for the seven burst-pause features. While moderate correlations exist between metrics such as burst count and pause rate, the lack of high redundancy supports the inclusion of all features in the model.

5.2 Classification Models

5.2.1 Mouse Dynamics Approach

To assess whether increased model complexity improves performance, ten models across these categories were implemented. The performance results of each can be seen in Figure 7

Logistic Regression serves as a simple linear baseline, while K-Nearest Neighbours (KNN) provides a distance-based comparison for the low-dimensional feature space.

Random Forest, Gradient Boosting, and XGBoost were selected for their ability to capture non-linear interactions and their proven efficiency on structured tabular data.

Support Vector Machines (SVM) with an RBF kernel model flexible decision boundaries, and Isolation Forest evaluates bot detection as an unsupervised anomaly detection problem.

Two Multilayer Perceptron (MLP) variants test whether over-parameterized models offer advantages. Additionally, Long Short-Term Memory (LSTM) networks process raw trajectories to capture temporal dependencies missed by aggregated features.

Figure 7: Performance heatmap for all 10 mouse models across multiple metrics. Most models hit the ceiling imposed by the dataset artefact

The near perfect performance of most models is driven by the standard deviation of speed (std speed). As mentioned previously, bot trajectories use fixed step sizes, resulting in near-zero variability, while human movement is significantly more diverse. This creates a trivial classification boundary, allowing even simple models like Logistic Regression to achieve an AUC of 1.00. More complex architectures offer no additional benefit because the classes are already perfectly separable via this single feature.

XGBoost achieves AUC = 1.0 on the holdout set, consistent with all other classical models. The slight CV F1 dip (0.994 ± 0.012) reflects one or two cross-validation folds where the regularised tree splits land slightly differently in the 3D feature space, but these edge cases do not appear in the held-out test performance.

Isolation Forest fails completely (AUC around 0.30). This is because Isolation Forest is an unsupervised method — it

does not know which class is the "anomaly". It labels the most isolated samples as anomalies. In a balanced dataset where bots cluster tightly (zero speed variation), the algorithm may actually identify bots as the majority pattern and incorrectly label humans as anomalies. An AUC of 0.30 means its predictions are systematically reversed — it is below the 0.5 random baseline.

The MLP required early stopping to be disabled. With only 120 training sessions and a network with over 10,000 parameters, the internal validation split (12 samples) was too small to give a reliable signal. When early stopping was active, the network stopped training too early and reached near-random performance (AUC of 0.49). This is a sign that the model architecture is too complex for this dataset — a smaller model would be the correct fix.

Additionally, to test these findings beyond binary detection, the models were evaluated on a three-class problem: humans, moderate bots, and advanced bots. This setup assesses a model's ability to distinguish between varying levels of bot sophistication. The following charts illustrate how each model navigates these more nuanced classification boundaries. The results of that additional analysis can be seen in Figure 8.

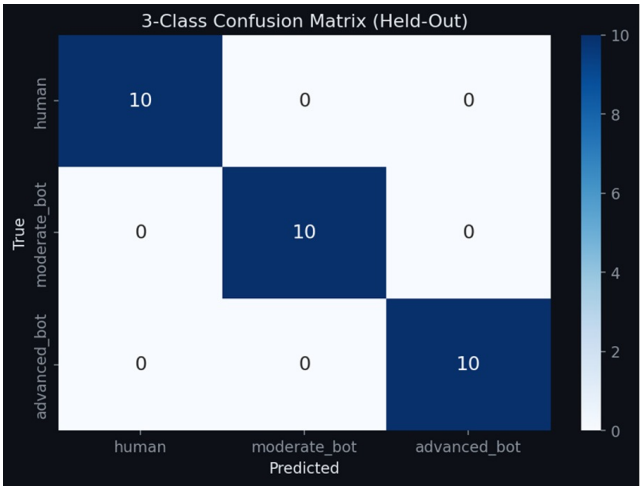

Figure 8: Confusion matrix for 3-class classification (human vs moderate bot vs advanced bot)

5.2.2 Keystroke Dynamics Approach

We evaluated eight models on keystroke dynamics: six classical machine learning models and two neural networks. The classical models and the Multilayer Perceptron (MLP) utilized seven pre-computed summary features, while the Long Short-Term Memory (LSTM) network processed raw Hold Time (HT) and Flight Time (FT) sequences directly.

To assess generalization, we performed in-distribution testing using unseen participants performing the same task as the training set (English opinion writing). This setup isolates the model's ability to identify behavioral patterns rather than specific individuals. In this controlled environment, performance was robust, with AUC scores ranging from 0.83 to 0.93.

Figure 9: Performance heatmap of all eight keystroke models across Accuracy, Precision, Recall, F1, AUC, and Average Precision. The MLP leads in AUC (0.9252), while the SVM demonstrates superior inference speed and OOD robustness.

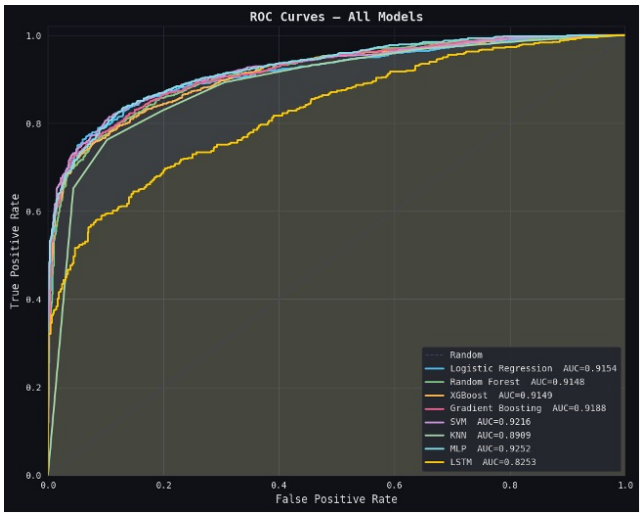

Figure 10: ROC curves for all eight keystroke models on the in-distribution test set. The curves illustrate strong performance across the ensemble, with most models clustering near the top-left quadrant.

Despite being a neural network, the MLP outperformed all other models. Its success stems from its ability to capture complex non-linear interactions within the seven summary features without suffering from over-parameterization. Conversely, the LSTM performed poorest. Despite having access to raw temporal sequences, training proved unstable.

The SVM emerged as the top classical performer. In a low-dimensional 7D space, the RBF kernel excels at find-

ing clean boundaries between burst-pause patterns. Models ranked 3rd through 6th (Boosting, LR, XGBoost, RF) performed nearly identically, within a 0.004 AUC margin. This suggests that in a low-dimensional feature space, the choice between these standard classifiers is less critical than the quality of the features themselves.

KNN model's lower performance highlights a specific security weakness. Because it relies on Euclidean distance, it can be deceived by sophisticated bots (e.g., CalibratedBot) that generate feature vectors close to the human population mean. Confusion matrices confirm that this proximity-based classification fails when bot behavior overlaps significantly with human distribution.

5.2.3 Experiment: Cross-Text Evaluation

To assess real-world viability, models were subjected to a cross-text experiment where training on English opinion writing (PROSODY) was tested against Spanish healthcare typing (LSIA) and fixed password entry (KM). These datasets introduce variations in language, syntax, and cognitive load that significantly alter typing rhythms and pause behaviours. As shown in Table 4, all models experienced a performance decline, revealing that high in-distribution results are heavily influenced by the shared characteristics of the training and testing environments.

The results highlight a clear trade-off between peak accuracy and robustness. The SVM proved most resilient, maintaining an AUC of 0.8111 (a drop of only 0.110), while the other models suffered the steepest decline to a minimum of 0.60 (LSTM).

Ultimately, the discrepancy between in-distribution and Out-of-Distribution (OOD) results has significant implications for deployment. For production environments where input is unpredictable, the SVM is the superior candidate, balancing stable cross-text performance with rapid inference speed. MLP achieves the highest in-distribution AUC (0.9252) but drops significantly cross-text (0.6797). LSTM is not recommended: it ranks last in both settings and costs 177 seconds to train with 286 MB peak memory.

6 Multi-modal System

This section describes the fusion approach to using the best models previously identified. The goal is to understand whether the strongest individual detectors combined can produce a better or more interpretable system.

To ensure a controlled and independent evaluation, all keystroke models in this section were retrained from scratch rather than reusing the previous results. This deliberate choice serves two purposes: it confirms that the reported performance figures are reproducible across independent training runs, and it ensures that the models evaluated here were trained and tested within the same pipeline as the fusion system, eliminating any risk of data-handling inconsistencies in between. The same parameters, feature set, and participant-level split were used.

6.1 Methodology and Experimental Design

This study uses **score-level (late) fusion**, which basically means we take the probability scores from each subsystem and combine them into one final classification. Here are the different methods we used:

- **Average:** We took the arithmetic mean of the two scores. This treats both the mouse and keystroke channels as being equally reliable.
- **Max (OR logic):** This flags a session as a bot if *either* channel identifies it as one. This gives us high recall, but it also leads to a higher false-alarm rate.
- **Min (AND logic):** This only flags a session as a bot if *both* channels agree. This keeps the false-alarm rate low, but it usually results in lower recall.
- **AUC-weighted average:** We weighted each channel based on its individual AUC. This way, the stronger, more accurate channel has more influence on the final score.
- **Meta-learner:** We trained a small Logistic Regression model to figure out the best linear combination of the two scores. For this, we used 70

Experimental Limitation: Since there aren't any public datasets that have both keystroke and mouse data from the same person, we couldn't test fusion using actual joint sessions. To get around this, we used bootstrap pairing (N = 300 pairs per class) to randomly match keystroke and mouse scores from different test users. This allowed us to create synthetic "dual-modality" sessions.

It is worth noting that in a real-world scenario, there's likely a correlation between how someone types and how they move a mouse, since both are controlled by the same nervous system. Our pairing method removes that natural correlation. Because of this, these fusion results should be

Model	In-dist AUC	OOD AUC (LSIA+KM)	AUC Drop	OOD Rank
SVM	0.9216	0.8111	−0.110	1st — best generaliser
KNN	0.8909	0.7356	−0.155	2nd
XGBoost	0.9149	0.7121	−0.203	3rd
Gradient Boosting	0.9188	0.7026	−0.216	4th
Random Forest	0.9148	0.6916	−0.223	5th
Logistic Regression	0.9154	0.6880	−0.227	6th
MLP (Keras)	0.9252	0.6797	−0.246	7th
LSTM 64/32	0.8253	0.6052	−0.220	8th — worst generaliser

Table 4: Performance metrics comparison of models across In-distribution and Out-of-Distribution (OOD) testing scenarios.

seen as a general proof of concept rather than an exact estimate of how the system would perform in a live environment.

We evaluated all 25 possible pairings of the five keystroke models and five mouse models using average fusion. To do this, we used bootstrap pairing to create 300 synthetic sessions for each class. The heatmap below displays the resulting AUC for every combination.

All 25 configurations reached an AUC of 1.0 or very close to it. This result was expected; since every mouse model already achieves an AUC of 1.0 on the M4D dataset, combining them with any keystroke model using average fusion naturally results in a near-perfect fused score.

6.2 Results and Partial Attack Analysis

We assumed that the most realistic threat model is a partial attacker: a bot that has cracked one biometric channel but not the other. We simulate four scenarios using the primary pair (KS SVM + Mouse SVM) with average fusion:

- **Full human:** Both channels produce genuine human scores. Measures the false-alarm rate: legitimate users incorrectly flagged as bots.
- **Partial A — bot mouse, human keystrokes:** The attacker uses a scripted mouse driver but successfully mimics human typing. The mouse channel score is a genuine bot score; the keystroke score is a genuine human score.
- **Partial B — bot keystrokes, human mouse:** The attacker uses a mechanical typing pattern but successfully mimics human mouse movement. The keystroke channel score is a genuine bot score; the mouse score is a genuine human score.
- **Full Bot:** Both channels produce genuine bot scores. Measures the system recall: bots that are correctly identified.

Initial results showed near-perfect performance (AUC $\approx$ 1.0) across all configurations. However, this is largely a byproduct of the mouse dataset's inherent separability rather than the fusion logic itself. To gain more meaningful insights, we conducted a **Partial Attack Analysis** simulating a sophisticated attacker who successfully spoofs one modality but fails the other.

Partial A (bot mouse, human keystrokes) is caught 100% of the time by average fusion. This reflects the dominance of the mouse channel: even a single channel detecting a bot is enough to push the average score above the decision threshold when one channel is a perfect detector (AUC = 1.0). Partial B (bot keystrokes, human mouse) is caught only 26% of the time by average fusion. This is the harder case: the human mouse score drags the fused score downward, and the keystroke bot score alone is not strong enough to push the average above 0.5 in most cases. Max (OR) fusion catches 88% of Partial B attacks — at the cost of a much higher false-alarm rate (22% on full-human sessions). Min (AND) fusion essentially ignores Partial B entirely (0% detection) because it only fires when both channels agree. The false-alarm rate under average fusion is just 1%: genuine human sessions are almost never misclassified. Full-bot recall is 100% under average fusion, meaning no full bots escape detection.

6.3 Compute Efficiency

Because this section targets deployment, we also measured the compute cost for every model. The performance metrics across all 10 models (5 mouse + 5 keystroke) are visualized in the Full Compute Dashboard (Figure 11). We evaluate three key metrics: training time, inference latency (averaged over five runs per sample), and peak memory usage during training (measured via `tracemalloc`).

For a real-time bot detector, the most important factor is how fast the model can make predictions (inference latency).

Scenario	Average	Max OR	Min AND
Full Human	0.01	0.22	0.00
Partial A (bot mouse)	1.00	1.00	0.20
Partial B (bot KS)	0.26	0.88	0.00
Full Bot (No Spoof)	1.00	1.00	0.91

Table 5: Partial attack flag rates for KS SVM + Mouse SVM under three fusion strategies. ”Flag rate” means the fraction of sessions that the system marks as bots.

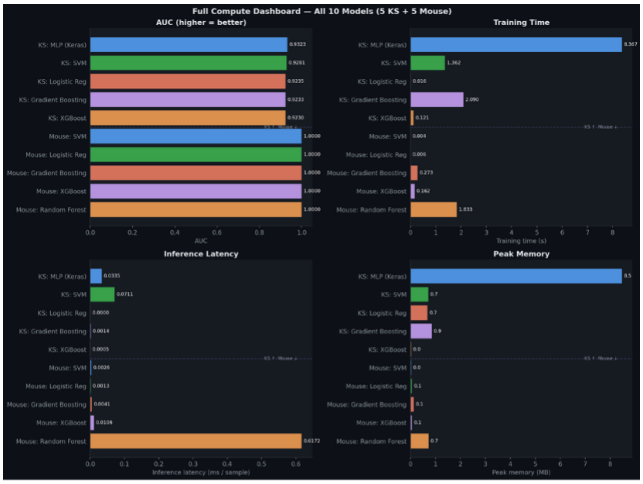

Figure 11: Full Compute Dashboard: Efficiency metrics for KS and Mouse models.

As shown in the dashboard, all our models easily stay under the 1 ms per-sample limit. Logistic Regression and XGBoost are extremely fast, with latencies so low they are basically negligible (between 0.0000 and 0.0005 ms). Even though SVM is the slowest of the classical models at 0.07 ms per keystroke session, it is still way within our budget. The MLP model has a tiny bit of extra Keras overhead (0.03 ms) compared to the others, and mouse inference is also super fast across all architectures (¡ 0.7 ms, with SVM at only 0.003 ms).

The recommended setup for deployment is **KS SVM + Mouse SVM**. This specific pairing offers the best balance between performance and efficiency. It has a total inference latency of only 0.075 ms per dual-channel session, a combined training time of about 1.4 s, and a peak memory footprint of roughly 0.7 MB. These metrics show that the system is definitely fast and light enough for real-time, high-traffic evaluation on the server side.

6.4 Discussion and Limitations

The fusion of keystroke and mouse dynamics demonstrates a robust defense mechanism, raising the minimum security guarantee by requiring an attacker to bypass two distinct biometric channels. However, several critical limitations impact the interpretation of these results:

- **Synthetic Correlation Gap:** Due to the lack of a joint dataset, results were derived from pairing of the González & Calot (keystrokes) and M4D (mouse) datasets [8], [5]. This removes natural correlations in human behavior (e.g., shared fatigue or cognitive load) across modalities. The true fusion AUC may vary when applied to joint data such as the BeCAPTCHA dataset [1].

- **Mouse Modality Dominance:** The near-perfect performance of mouse models on the M4D dataset (AUC = 1.0) saturates the fusion results. While this provides high security, it makes it difficult to distinguish between different fusion strategies based on AUC alone.

- **The Partial B Vulnerability:** A sophisticated attacker who successfully mimics human mouse movement while using mechanical keystrokes (Partial B) is detected only 26% of the time under average fusion. Improving this detection rate requires either a more discriminative keystroke model or a more aggressive decision threshold.

- **Threshold and Calibration:** All results utilize a fixed decision threshold of 0.5. In production environments, this would need to be calibrated against a specific False Positive Rate (FPR) budget. Furthermore, because the Logistic Regression meta-learner was trained on synthetic pairs, it may face calibration risks if the true joint score distribution differs significantly from the bootstrap estimate.

In summary, the KS SVM + Mouse SVM configuration with average fusion establishes a high-performance, deployment-ready pipeline for real-time bot detection. With a total inference latency of only 0.075 ms, the system is objectively suitable for high-traffic production environments

where non-intrusive security is paramount. While mouse dynamics currently offer the stronger signal, this multi-modal architecture effectively increases the ”cost of attack” by forcing adversaries to bypass two independent behavioral channels, providing a robust and scalable foundation for modern behavioral authentication.

7 Conclusion

This research investigated whether behavioral patterns derived from keystroke and mouse dynamics can reliably distinguish genuine human interaction from malicious simulated behavior. Through systematic feature engineering, model evaluation, and multi-modal fusion, several key findings emerged.

For mouse dynamics, exploratory analysis revealed that M4D dataset's bot trajectories rely on fixed pixel step sizes, making features such as standard deviation of speed highly discriminative. As a result, even simple linear classifiers achieved near-perfect separation (AUC = 1.0), which means that the classification performance reflects a dataset artifact rather than a genuine modeling challenge. Future work should evaluate mouse-based detection against more sophisticated adversaries that introduce realistic speed and trajectory variability.

For keystroke dynamics, the burst-pause feature engineering approach proved effective at capturing the cognitive rhythm of human typing. The MLP achieved the highest in-distribution AUC (0.9252), but the SVM demonstrated superior generalization in the cross-text evaluation, maintaining an AUC of 0.8111 when tested against unseen languages and task formats. This finding underscores a practical trade-off: models that excel in controlled conditions do not necessarily generalize to the diverse input environments encountered in deployment.

The multi-modal fusion analysis demonstrated that combining keystroke and mouse channels raises the cost of attack by requiring an adversary to simultaneously spoof two independent behavioral modalities. The recommended KS SVM + Mouse SVM configuration with average fusion achieved strong detection performance with a total inference latency of only 0.075 ms, making it suitable for real-time environments. However, the partial attack analysis exposed an important vulnerability: when an attacker successfully mimics human mouse movement while using mechanical keystrokes, detection rates under average fusion drop to just 26 percent. Addressing this asymmetry remains an important direction for future research, potentially through adaptive threshold tuning or channel-specific weighting strategies.

Several limitations should be acknowledged. The absence of a joint dataset containing both keystroke and mouse data from the same participants required the use of bootstrap pairing, which removes natural cross-modal correlations present in real human behavior. Additionally, the simplicity of the bot models in the M4D dataset limits the conclusions that can be drawn about detection robustness against state-of-the-art adversaries. Future studies should prioritize the collection of joint multi-modal datasets and the development of more realistic synthetic attack models, including those based on generative adversarial networks and reinforcement learning.

Despite these limitations, this work contributes a framework for behavioral bot detection that spans feature engineering, model selection, cross-domain generalization testing, and multi-modal fusion. The findings reinforce the value of behavioral biometrics as a non-intrusive and continuous security layer and highlight the importance of evaluating such systems not only under controlled conditions but also against realistic, partial-knowledge attack scenarios. The github repositories are available at [9] and [10].

References

[1] A. Acien, A. Morales, J. Fierrez, and R. Vera-Rodriguez, “BeCAPTCHA-Mouse: Synthetic Mouse Trajectories and Improved Bot Detection,” *arXiv preprint arXiv:2005.00890v2*, Mar. 2021.

[2] J. Xiong and A. K. Belman, “Multi-Modal Adversarial Activity Detection Using Keyboard and Mouse Dynamics,” in *Proceedings of the 2025 IEEE World AI IoT Congress (AIIoT)*, 2025, pp. 340–345.

[3] S. Sadeghpour and N. Vlajic, “ReMouse Dataset: On the Efficacy of Measuring the Similarity of Human-Generated Trajectories for the Detection of Session-Replay Bots,” *Journal of Cybersecurity and Privacy*, vol. 3, no. 1, pp. 95–117, 2023.

[4] D. Stefan, X. Shu, and D. Yao, “Robustness of Keystroke-Dynamics Based Biometrics Against Synthetic Forgeries,” *Computers & Security*, vol. 31, no. 1, pp. 109–121, Feb. 2012.

[5] N. González, “Dataset of Human-written and Synthesized Samples of Free-Text Keystroke Dynamics

to Evaluate Liveness Detection Methods," *Mendeley Data*, vol. 1, Aug. 2022.

[6] N. González, E. P. Calot, J. S. Ierache, and W. Hasperué, "Towards Liveness Detection in Keystroke Dynamics: Revealing Synthetic Forgeries," *Systems and Soft Computing*, vol. 4, p. 200037, Dec. 2022.

[7] C. Iliou, T. Kostoulas, T. Tsikrika, V. Katos, S. Vrochidis, and I. Kompatsiaris, "Detection of Advanced Web Bots by Combining Web Logs with Mouse Behavioural Biometrics," *Digital Threats: Research and Practice*, vol. 2, no. 3, pp. 1–26, Jul. 2021.

[8] K. Garg, "m4d-mouse-dynamics," *Kaggle*, 2025. [Online]. Available: `https://www.kaggle.com/datasets/donutkashvi/m4d-mouse-dynamics/data`. [Accessed: Mar. 13, 2026].

[9] K. Nurlybekov, "mousedynamics," *GitHub Repository*, 2025. [Online]. Available: `https://github.com/knurlybekov/mousedynamics`. [Accessed: Mar. 13, 2026].

[10] K. Nurlybekov, "keyboarddynamics," *GitHub Repository*, 2025. [Online]. Available: `https://github.com/knurlybekov/keyboarddynamics`. [Accessed: Mar. 13, 2026].